Web & Mobile Developer

Ju-cheol Park

Full-stack AI document analysis and settlement platforms, with web applications and Android integration for business workflows.

Experience

Triphos Co., Ltd. / 2026.02 ~ Present

Scope

Built web interfaces and server APIs, and integrated Android app features

jukrap628@gmail.com github.com/jukrap www.linkedin.com/in/jukrap/ jukrap.vercel.app/en

Selected work

01 AI Document and Data Analysis Platform

Built document search and answer evaluation. Removing repeated calculations reduced median document-analysis processing from 2,969ms → 672ms in a test with fixed model responses.

02 Settlement ERP and Mobile App

Built the ERP and connected mobile workflows across the stack, including concurrent-edit recovery, batch Excel imports, notifications and document downloads.

03 Logistics Operations Web and Printing App

Built the logistics frontend and Android printing app. Reduced the initial JavaScript file by about 74% and corrected clipped or overlapping addresses using physical labels.

04 Stock Operations Admin Web

Built the complete React frontend against MSW mock APIs, including shared tables, filters and query and editing state.

Next, I will introduce daily information and privacy apps, an internal documentation tool, further company work, and personal and team projects.

Company work 01

AI Document and Data Analysis Platform

Owned full-stack development of a Gemma 4 platform for document search and question answering. Improved retrieval of surrounding context, answer validation and Ragas evaluation.

Period

2026.09 ~ Present

Scope

Full-stack development

Technologies React, TypeScript, Tailwind CSS, Vite, TanStack Query, Zustand, Python, Django, MariaDB, SQLGlot, Gemma 4, Ragas, tokenizers, Vitest

Context

Matched passages lacked conditions and exceptions from nearby paragraphs. Questions with several parts sometimes returned only source text. Long-answer evaluation exceeded the model input limit, while repeated token counting slowed document analysis.

Implementation

Retrieve the surrounding context

Included nearby paragraphs and referenced tables with each search result. Supplied original text and sources during final answer generation to retain relevant conditions and exceptions.

Evaluate long answers in batches

Used Ragas to check whether the answer was supported by the document. Split long answers into batches that fit the model input limit and checked item IDs for missing or duplicate evaluation results.

Answer each part and preserve source quotations

Fixed explanation requests being misclassified as table requests. Collected item-level answers and evidence IDs, retained supported answers and copied quotations directly from the source.

Remove repeated calculations

Removed repeated token counting during table merging and reused the results of pages already analyzed within the same request.

Show the answer before evaluation finishes

Ran Ragas evaluation asynchronously after saving the answer. Deferred the next evaluation request while a new user question was being processed.

Outcome

Completed long-answer evaluations that had previously exceeded the input limit. Reduced document-analysis time by removing repeated calculations, and let users read answers before quality evaluation finished.

Median document-analysis processing time

2,969ms → 672ms

Compared application processing with fixed model responses: 20 runs per condition after warm-up, excluding model and network wait time.

Completed evaluation of a long answer within the input limit

25 items evaluated

Evaluated a Gemma 4 answer about a real document in 5 batches. The largest request used 8,164 tokens, below the 8,192-token input limit.

No need to wait for quality evaluation

Display the answer first, then evaluate it separately

Users can read the answer before Ragas finishes. The next evaluation request is deferred while a new user question is being processed.

Company work 02

Settlement Operations Platform

Owned full-stack development of an ERP covering vehicles, drivers, transport fees, billing and settlement. Built editing and Excel-import workflows that preserve user input and finalized settlement records.

Period	Scope	ERP web, API and mobile contribution
2026.07 ~ 2026.09	Full-stack development	3,079 files / 424,000 lines

Technologies React, TypeScript, Tailwind CSS, Vite, React Router, Zustand, Spring Boot, MyBatis, MariaDB, Vitest

Context

Concurrent edits could overwrite someone else's saved changes. Excel imports also needed to avoid partial writes and duplicate records after retries, while updates to current records had to preserve finalized settlement amounts.

Implementation

Retain input when concurrent edits conflict

Built shared tables with keyboard cell and dialog controls. Checked the latest server version before saving. On a conflict, retained the draft and compared the original, latest and entered values, letting the user choose values before explicitly saving again.

Correct, revalidate and save Excel imports together

Highlighted invalid cells for correction and revalidation. The server retained the validated data, then rechecked permissions and referenced records before saving. Failed batches rolled back together, and repeated requests returned their existing result.

Separate current records from finalized settlements

Separated current vehicle and driver assignments from historical transactions. Changes to loan and insurance records updated draft or rejected charges while preserving amounts that had already been approved or settled.

Make work logs understandable to administrators

Reorganized technical logs around affected records, changes and input-error reasons. Included the record and attempted input when a save conflicted with another update.

Outcome

Reproduced concurrent saves in the development environment and compared the interface, API, database and logs. Verified that the latest data and draft were retained and only selected values were saved again. Added batch rollback and repeat-request handling to Excel imports, with work and error details available in administrator logs.

Company work 02 / Mobile

ERP-connected Mobile Workflow App

Owned most development of the ERP-connected mobile work app, excluding the chat system. Built React business screens and Spring Boot APIs and connected them to notifications and file downloads in the Flutter app.

Period
2026.07 ~ 2026.09

Scope
Full-stack development · excluding the chat system

Technologies React, TypeScript, TanStack Query, TanStack Router, Spring Boot, MariaDB, Flutter, Dart, Firebase Cloud Messaging

Context

Notifications could arrive out of order and restore outdated content after an edit or deletion. Concurrent receipt writes caused a database deadlock, and browser-only file URLs could not be passed directly to the app for downloads.

Implementation

Keep delayed notifications from replacing current state

Stored notification versions and deletion state to ignore older events arriving late. Preserved read status when content changed and refreshed details that the user already had open.

Fix concurrent receipt writes

Replaced an update-then-insert sequence with a single UPSERT while preserving the first seen and read timestamps. Reproduced the original deadlock in an isolated MariaDB database and verified concurrent requests after the change.

Issue document URLs the app can download

Replaced browser-memory file URLs with short-lived signed URLs issued when the user requests a download. Rechecked the signature, expiry and current access rights when serving the file, without passing login tokens to the native download feature.

Connect inquiry handling and app notifications

Linked mobile inquiries to ERP processing and history shown in the app. Explained completed or conflicting updates while retaining the user's input. Connected FCM notifications and navigation to the relevant detail screen in Flutter.

Outcome

Prevented older notifications from reversing edits or deletions and resolved deadlocks in concurrent receipt writes. Compared a browser-downloaded PDF with its source and verified receiving a notification and opening the related list and content on an Android emulator.

Company work 03 / Web

Logistics Operations Web

Built the complete frontend for a logistics web app covering search, reservations, Excel imports and label printing. Integrated server APIs and PC printers, and connected the same business screens to label printing in an Android app.

Period	Scope	Source contribution
2026.04 ~ 2026.06	Complete frontend development	403 files / 63,000 lines

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, XLSX, Vitest

Business interfaces and printing

Shared business screens and API integration

Built shared tables, forms and modals and connected search, reservations, Excel previews and print requests to server APIs. Used a common print-data structure for PC and Android printer handling.

Fit long addresses using physical print feedback

Checked physical labels for clipped or overlapping addresses. Adjusted line wrapping by text width, overall font size and line spacing to fit the available label area.

Load only the code needed on entry

The initial JavaScript file included pages and Excel code that were not needed on entry. Long addresses could also be clipped or overlap on physical labels even when the screen preview looked correct.

Analyzed the bundle to identify code that was unnecessary on entry. Loaded pages when users entered them and Excel libraries when they started an upload or download.

Bundle size change

Initial JavaScript file size

2,405.50 → 616.60 kB

About 74% smaller in the May 2026 build outputs

Gzip-compressed size

815.10 → 204.38 kB

About 75% smaller for the compressed initial JavaScript file

Scope of code splitting

Splitting authentication and API initialization reduced the initial file further, but complicated initialization order and loading on the first request. Weighed the extra reduction against maintenance cost and limited lazy loading to pages and Excel libraries.

Company work 03 / Android

Logistics Label Printing App

Connected web label-print requests to native modules and Bluetooth printers in an Android app. Built device selection, connection checks and the print-request flow.

Period	Scope	Source contribution
2026.04 ~ 2026.06	Android app development	81 files / 7,000 lines

Technologies Expo, React Native, Expo Router, WebView, Android native module, TypeScript, Bluetooth

Context

The printer appeared in the list and could be selected, but connection failed. Finding the cause required tracing app permissions and actual printer SDK calls alongside the web request.

Implementation

Label-printing path

01	02	03
WebView	**Android**	**Bluetooth**
Label data and print request	Bridge, permissions, device connection	Printer SDK and label output

Convert web requests into printer data

Passed WebView print requests through a native bridge and converted them into printer data. Let users inspect the items and select the target device in the app.

Trace SDK calls to resolve a permission failure

Device logs showed that the SDK cancelled Bluetooth discovery immediately before connecting. Requested the discovery permission required by that call alongside the connection permission, then verified physical label output.

Distinguish request acceptance from physical printing

Changed completion messages so an SDK acknowledgement did not imply that a label had finished printing. Showed that the print request had been sent.

Outcome

Verified permission requests, Bluetooth connection and physical label output on an Android 16 device. Made it possible to trace connection failures across web requests, app permissions and SDK calls.

Company work 04

Stock Operations Admin Web

Built the complete React frontend for account, stock and order administration using MSW mock APIs. Shared table components and query-state handling across recurring screens.

Period	Scope	Source contribution
2026.02 ~ 2026.03	Complete frontend development	182 files / 18,000 lines

Technologies React, TypeScript, Vite, React Router, TanStack Query, Zustand, MSW

Shared tables, filters, and modals

Built shared list, search, detail and editing components for account, stock and order screens. Added column pinning, resizing and reordering to tables, and implemented queries and input interactions with MSW mock APIs.

Applying order-history filters

Kept account and stock selections separate from the conditions applied to the list. Editing a filter retained the existing results; pressing Search applied the submitted conditions to the TanStack Query key.

Repeat searches and load more

Search again with the same filters

Reset the existing query to reload from the first page.

Guard load-more requests

Skipped additional requests when no next page remained or a request was already in progress.

Company work 05

Weather in a Daily Information App

Added public-data weather features to an existing Android app, implementing the WebView interface, server-side API integration, and caching.

Period
2026.06 ~ 2026.07

Scope
Frontend and backend feature development

Technologies Spring MVC, JSP, jQuery, Java, Android

Weather data across separate APIs

Weather, air quality, and alerts had different regional coverage and update schedules. Fetching everything on every visit repeated requests for the same area and could hold up available information while a slower response arrived.

Caching by region and data type

Reuse data for requested regions

Cached responses by region and data type, with expiry periods matched to each type.

Share in-flight requests

Reused an in-flight result for the same cache key rather than issuing another external API call.

Show core information first

Displayed current conditions and the core forecast first, then filled in air quality and alerts as responses arrived. Delays or failures in supplementary requests did not hide weather data already received.

Handling stale information

When an external request failed, allowed stale values could remain visible. An expired “no alerts” response was marked unavailable rather than treated as current. Previously retrieved active alerts were labelled as based on the last retrieval.

Company work 06

Privacy Check App

Added email-breach lookup and deepfake image checks to an existing Android app. Connected inspection screens, server calls to external APIs, and Android image selection from input through results.

Period
2026.06

Scope
Frontend and backend feature development

Technologies Android, Cordova, Spring MVC, jQuery, Java

Turn external inspection APIs into app features

Implemented server calls to external APIs for submitted emails and images, translating responses and errors for the interface. API credentials remained on the server; the app received the results and status needed for display.

From image selection to inspection

Input validation and preview

Validated the selected image's file type, size and resolution, and displayed a preview before submission.

Android file selection

Opened the Android file picker for image or file requests from the WebView and returned the selected file to the WebView.

When the user selects another image

A previous file read can finish after the user selects another image. Assigned a token to each read and checked it on completion so stale results would not overwrite the new preview.

Company work 07

AI-assisted Project Documentation Web App

Built an internal web tool for drafting and reviewing documents from repositories and project kickoff material. Implemented screens and server APIs for collecting sources, reviewing requirements, editing tables and exporting documents.

Period
2026.04

Scope
Frontend and backend development

Technologies Node.js, TypeScript, React, AI API, xlsx, Vitest

Review source material before drafting

Split the interface into review stages for proposed requirements and open questions before drafting final documents. Enabled functional specifications after the requirements document was ready, and screen specifications after their prerequisite documents were ready.

Review tables and revise selected content

Edit document sheets

Made requirements, functional specifications and screen specifications editable as separate sheets. Exported documents from the tables as edited by the user.

Targeted AI revisions

Sent the selected sheets or cells together with surrounding content. Distinguished the revision target from content to preserve so users could change the selected part without regenerating the entire document.

Handle partial generation failures

Tracked generation status per document so completed documents remained available for review after another failed. Supported regenerating and exporting selected documents without restarting the entire set.

Company work

Further company work

Feature additions to existing services and updates for Android versions and field use.

08 Member and Inquiry Management Web and App

Period
2026.09

Platform
Web / Android

Implementation

Added member search, editing, duplicate merging, inquiries and attachments to existing web and Android apps. Checked member roles and authorship on the server and connected file selection and image previews in the app.

Technologies Spring MVC, Java, MyBatis, Vue 2, MariaDB, Android, Kotlin

09 Parcel Delivery App

Period
2026.08

Platform
Web / Android

Implementation

Updated map-marker selection and return-to-map behavior. Retained delivery-completion requests and photos offline, restored requests after app restarts and compared them with completed server records before retrying.

Technologies React, TypeScript, React Native, Expo, Spring Boot, SQLite

10 Chart Preview Page

Period
2026.03 ~ 2026.04

Platform
Web

Implementation

Built chart settings, data connections and previews for a website under development. Connected chart-specific data fields and reduced unnecessary recreation to limit flicker and scroll movement.

Technologies React, TypeScript, Vite, Chart.js, WebGL/GLSL (OpenGL-style), OGL (WebGL library), MSW, Vitest

11 Logistics Receiving and Shipping PDA App

Period
2026.03 ~ 2026.04

Platform
Android

Implementation

Added receiving filters, quantity entry and QR/barcode scanning to an existing PDA app. Separated validation, duplicate handling and quantity rules by work mode, and used device input and logs to distinguish external validation failures from app errors.

Technologies Android Java, Gradle/AGP, Scanner SDK

12 Legacy Shipping and Inventory App

Period
2026.03

Platform
Android

Implementation

Aligned compatible development-tool versions to restore builds of a legacy Android app. Updated OS-specific permissions, file access and back navigation and checked behavior on devices running older OS versions.

Technologies Android Java, Gradle/AGP, RxJava, FileProvider

Project 01

C. Donghae

Built a map-based web service for Donghae Line passengers to find live transport and nearby information.

Period

2024.10.04 ~ 2024.10.06

Role

Frontend developer / team of three

Technologies TypeScript, React, Next.js, Tailwind CSS, Google Maps, Storybook, AWS, GitHub Actions, Docker

Hackathon scope

The team had 72 hours to combine train, weather, and local-place data in one map with mobile interactions.

Implementation

Maps and API integration

Worked with two backend developers using Swagger documentation to connect APIs and build the main Google Maps screens.

Mobile map interaction

Built a bottom sheet so users could browse places while keeping the map visible. The sheet expands or collapses according to drag distance and velocity.





Award

Won the Busan Technopark President Award at DIVE 2024, placing third in the challenge track.

Project 02

ShareBBy

Developed Android support and community features for an app that helps people share hobbies and find companions.

Period

2024.04 ~ 2024.06

Role

React Native app development and Firebase integration

Technologies JavaScript, React Native, Firebase, Faster Image

Android support and image updates

Screens initially built for iOS needed Android support, and stale image caches prevented updated post images from appearing.

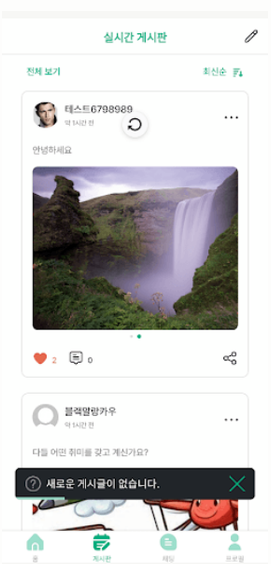
Implementation

Community features

Built post and comment creation, reading, editing and deletion, with location filters and sorting. Users can pull to refresh the list or scroll to load more posts.

Data and image handling

Documented Firebase relationships in an ERD, reproduced the caching problem, and adopted a maintained replacement library.





Release

Delivered Android screens and community features. The team released the app on the App Store in 2024.

GitHub

github.com/jukrap/archived-rn-ShareBBy

Project 03

AI Agent Playbook

Built a tool that helps AI coding tools refer to project rules and work history across sessions. Users can select skills to install, change the setup and recover it while preserving existing files.

Period

2026.06 ~ Present

Role

Solo developer

Technologies JavaScript, Node.js, MCP SDK, Zod, GitHub Actions

Context and user-file preservation

Working across repositories required shared records to remain distinct from local history. Updating skills also had to preserve user edits and avoid losing a working setup after a failed installation.

Implementation

Shared records and read-only MCP

Implemented search and read access to registered repositories' records through a CLI and read-only MCP. Split long results into pages and stopped continuation if the source changed, so old and new content would not be mixed.

Preserving edits and recovering installations

Checked which files the tool installed and whether they had changed to preserve user edits. Required all replacement skills to be valid before removing old entries, and rechecked for changes immediately before deletion. Kept backups and application records to restore the previous setup.

Release and verification

Published on npm and GitHub, with installation and usage documented in English and Korean. Tested installation conflicts, partial failures, recovery, query scope and continuation, and configured Windows/Ubuntu CI.

GitHub
github.com/jukrap/ai-agent-playbook

npm
www.npmjs.com/package/ai-agent-playbook

Project 04

Itzip

Led frontend work in a 15-person project for aspiring developers. Built blog and Markdown editing screens and configured tests, error monitoring and deployment infrastructure.

Period

2024.07 ~ 2025.06

Role

Frontend team lead / DevOps

Technologies TypeScript, React, Next.js, Tailwind CSS, React Spring, Jotai, Jest, Prisma, Storybook, Postman, Sentry, AWS, Jenkins, Docker

Editing and team development

The team needed a shared way to inspect component behavior and errors after deployment while building the editor.

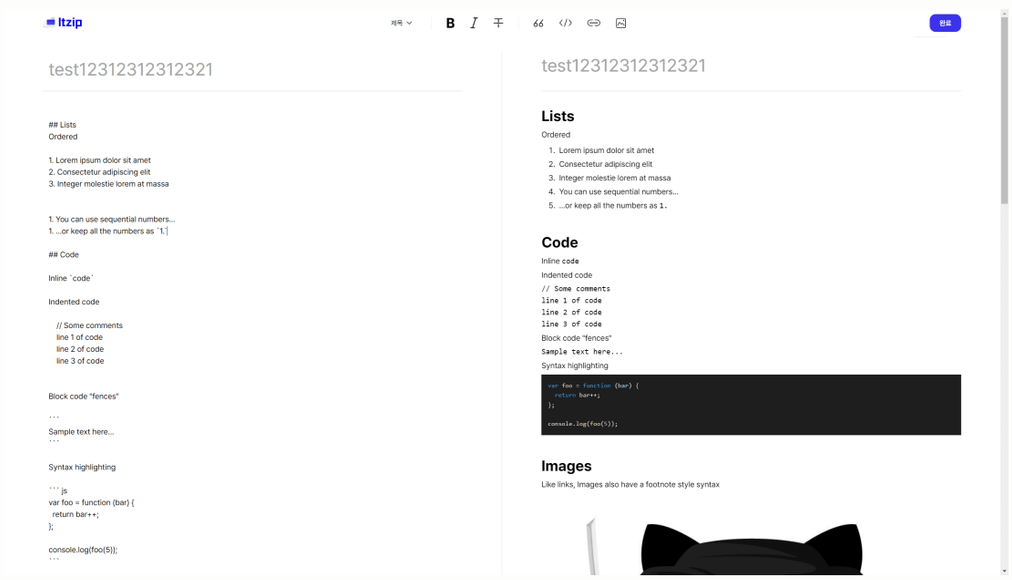
Implementation

Markdown editing and previews

Built a Markdown editor with live previews so writers could see the result as they typed. Supported project-specific syntax and loaded heavy post components on demand through dynamic imports.

Tests and error monitoring

Tested key behavior with Jest, documented components in Storybook, and configured Sentry for post-deployment errors.



Markdown editing and preview

Team adoption

Delivered blog and editor screens with shared tests, component documentation and error monitoring. Also configured the AWS runtime and deployment infrastructure with Jenkins and Docker.

GitHub

github.com/ITZipProject/itzip_front

Project 05

Posture Teacher

An Android app that finds body landmarks in camera frames, classifies sitting and plank posture, and records how long each posture is held.

Period

2022.07 ~ 2023.05

Role

Android developer / team of two

Technologies Java, Android, Jetpack, MediaPipe, SQLite

Camera analysis and interaction

Continuous frame analysis needed to remain separate from screen input, and MediaPipe had to be built for the Android project.

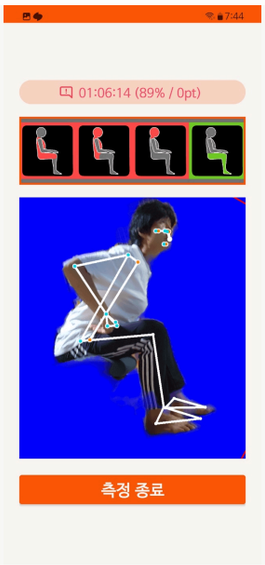
Implementation

MediaPipe integration and posture detection

Built a MediaPipe AAR on Ubuntu and used landmark angles and lengths to classify posture and measure hold times.

Separate frame analysis

Moved analysis to a separate thread so camera processing and screen input did not block each other.



Posture analysis and timing records

Measured result

During the project, camera frame-processing throughput (FPS) with MediaPipe was 5–10 times that of the earlier OpenCV implementation.

GitHub

github.com/jukrap/archived-Posture-Teacher

Profile

Skills, education, and awards

Skills

Languages

TypeScript, JavaScript, Java, Python, Dart

Web

React, Next.js, Vite, TanStack Query, Zustand, Django

Mobile / hybrid

React Native, Expo, Flutter, Android, Gradle

Testing / monitoring

Vitest, MSW, Storybook, Sentry, Ragas

Tools & Data

GitHub Actions, Jenkins, Docker, MariaDB, SQLite

Education & activities

Gyeongsang National University / Computer Science

Graduated 2023.02

GPA 3.64 / 4.5

Programmers Devcourse

2023.12 ~ 2024.05

Cloud Application Engineering, React and React Native

Programmers Devcourse / Assistant Mentor

2024.05 ~ 2024.09

Shared development resources and supported daily scrums and project reviews.

Awards

DIVE 2024 Global Data Hackathon

2024.10

Busan Technopark President Award / third in the challenge track

Gyeongnam Software Competition

2021.10

ESD HotDeal / Top Excellence Award

More

About
jukrap.vercel.app/en/about

Work
jukrap.vercel.app/en/work

Side Projects
jukrap.vercel.app/en/projects